HTTP Response Status Codes (Standard)

This article lists standard and notable non-standard HTTP response status codes. Standardized codes are defined by IETF as documented in Request for Comments (RFC) publications and maintained by the IANA. Other, non-standard values are used by various servers. The descriptive text after the numeric code - the reason phrase - is shown here with typical value, but in practice, can be different or omitted.

Standard codes

Status codes defined by IETF are listed below. Emphasized terms must, must not and should are interpretation guidelines as given by RFC 2119.

1xx informational response

An informational response indicates that the request was received and understood and is being processed. It alerts the client to wait for a final response. The message does not contain a body. As the HTTP/1.0 standard did not define any 1xx status codes, servers must not send a 1xx response to an HTTP/1.0 compliant client except under experimental conditions.

100 Continue

The server has received the request headers and the client should proceed to send the request body (in the case of a request for which a body needs to be sent, such as a POST request). Sending a large request body to a server after a request has been rejected for inappropriate headers would be inefficient. To have a server check the request's headers, a client must send Expect: 100-continue as a header in its initial request and receive a 100 Continue status code in response before sending the body. If the client receives an error code such as 403 (Forbidden) or 405 (Method Not Allowed) then it should not send the request's body. The response 417 Expectation Failed indicates that the request should be repeated without the Expect header as it indicates that the server does not support expectations (this is the case, for example, of HTTP/1.0 servers).

101 Switching Protocols

The requester has asked the server to switch protocols and the server has agreed to do so.

102 Processing (WebDAV; RFC 2518)

A WebDAV request may contain many sub-requests involving file operations, requiring a long time to complete the request. This code indicates that the server has received and is processing the request, but no response is available yet. This prevents the client from timing out and assuming the request was lost. The status code is deprecated.

103 Early Hints (RFC 8297)

Used to return some response headers before final HTTP message.

2xx success

A success status indicates that the action requested by the client was received, understood, and accepted.

200 OK

Standard response for successful HTTP requests. The actual response will depend on the request method used. In a GET request, the response will contain an entity corresponding to the requested resource. In a POST request, the response will contain an entity describing or containing the result of the action.

201 Created

The request has been fulfilled, resulting in the creation of a new resource.
202 Accepted
The request has been accepted for processing, but the processing has not been completed. The request might or might not be eventually acted upon, and may be disallowed when processing occurs.
203 Non-Authoritative Information (since HTTP/1.1)
The server is a transforming proxy (e.g. a Web accelerator) that received a 200 OK from its origin, but is returning a modified version of the origin's response.
204 No Content
The server successfully processed the request, and is not returning any content.
205 Reset Content
The server successfully processed the request, asks that the requester reset its document view, and is not returning any content.
206 Partial Content
The server is delivering only part of the resource (byte serving) due to a range header sent by the client. The range header is used by HTTP clients to enable resuming of interrupted downloads, or split a download into multiple simultaneous streams.
207 Multi-Status (WebDAV; RFC 4918)
The message body that follows is by default an XML message and can contain a number of separate response codes, depending on how many sub-requests were made.
208 Already Reported (WebDAV; RFC 5842)
The members of a DAV binding have already been enumerated in a preceding part of the (multistatus) response, and are not being included again.
226 IM Used (RFC 3229)
The server has fulfilled a request for the resource, and the response is a representation of the result of one or more instance-manipulations applied to the current instance.

3xx redirection

A 3xx status indicates that the client must take additional action, generally URL redirection, to complete the request. A user agent may carry out the additional action with no user interaction if the method used in the additional request is GET or HEAD. A user agent should prevent cyclical redirects.
300 Multiple Choices
Indicates multiple options for the resource from which the client may choose (via agent-driven content negotiation). For example, this code could be used to present multiple video format options, to list files with different filename extensions, or to suggest word-sense disambiguation.
301 Moved Permanently
The link target was moved such that the request and future similar requests should be redirected to the given URI. If a client has link-editing capabilities, it should update references to the request URL. The response is cacheable unless indicated otherwise. Except for a GET request, the body should contain a hyperlink to the new URL(s). Except for a GET or HEAD request, the client must ask the user before redirecting.This code is considered best practice for upgrading users from HTTP to HTTPS. Both Bing and Google recommend using this code to change the URL of a page as it is shown in search engine results, providing that URL will

permanently change and is not due to be changed again any time soon.

302 Found

Indicates that the resource is accessible via an alternate URL indicated in the Location header field. The HTTP/1.0 specification (which used reason phrase "Moved Temporarily") required the client to redirect with the same method, but popular browsers instead changed the request to GET. For this reason, HTTP/1.1 (RFC 2616) added two status codes: 303 which requires changing the request to a GET and 307 which preserves the original request type. Despite the greater clarity provided by this disambiguation, the 302 code is still used in web frameworks to preserve compatibility with browsers that do not support HTTP/1.1. As a consequence, RFC 7231 (the update of RFC 2616) changes the definition to allow user agents to rewrite POST to GET.

303 See Other (since HTTP/1.1)

If a server responds to a POST or other non-idempotent request with this code and a location header field, the client is expected to issue a GET request to the specified location. To trigger a request to the target resource using the same method, the server responds with 307 instead.

Use of this code has been proposed as one way of responding to a request for a URI that identifies a real-world object according to Semantic Web theory (the other being the use of hash URIs). For example, if http://www.example.com/id/alice identifies a person, Alice, then it would be inappropriate for a server to respond to a GET request with 200 OK, as the server could not deliver Alice herself. Instead, the server would respond with 303 to redirect to a URI that provides a description of the person Alice.

Sometimes, this code is used when providing an HTTP-based web API that needs to respond to the caller immediately, but continue executing asynchronously, such as a long-lived image conversion. The web API provides a status check URI that allows the client to check on the operation's status. When complete, the response may contain this status code and a redirect URI to the final result.

304 Not Modified

Indicates that the resource has not been modified since the version specified by the request headers If-Modified-Since or If-None-Match. In such case, there is no need to retransmit the resource since the client still has a previously-downloaded copy.

305 Use Proxy (since HTTP/1.1)

The requested resource is available only through a proxy, the address for which is provided in the response. For security reasons, many HTTP clients (such as Mozilla Firefox and Internet Explorer) do not obey this status code.

306 Switch Proxy

No longer used. Originally meant "Subsequent requests should use the specified proxy."

307 Temporary Redirect (since HTTP/1.1)

In this case, the request should be repeated with another URI; however, future requests should still use the original URI. In contrast to how 302 was historically implemented, the request method is not allowed to be changed when reissuing the original request. For example, a POST request should be repeated using another POST request.

308 Permanent Redirect

This and all future requests should be directed to the given URI. 308 parallels the behavior of 301, but does not allow the HTTP method to change. So, for example, submitting a form to a permanently redirected resource may continue smoothly.

4xx client error

A 4xx status code is for situations in which an error seems to have been caused by the client. Except when responding to a HEAD request, the server should include an entity containing an explanation of the error situation, and whether it is a temporary or permanent condition. These status codes are applicable to any request method. User agents should display any included entity to the user.

400 Bad Request

The server cannot or will not process the request due to an apparent client error (e.g., malformed request syntax, size too large, invalid request message framing, or deceptive request routing).

401 Unauthorized

Similar to 403 Forbidden, but specifically for use when authentication is required and has failed or has not yet been provided. The response must include a WWW-Authenticate header field containing a challenge applicable to the requested resource. See Basic access authentication and Digest access authentication. 401 semantically means "unauthenticated", the user does not have valid authentication credentials for the target resource.

402 Payment Required

Reserved for future use. The original intention was that this code might be used as part of some form of digital cash or micropayment scheme, as proposed, for example, by GNU Taler, but that has not yet happened, and this code is not widely used. Google Developers API uses this status if a particular developer has exceeded the daily limit on requests. Sipgate uses this code if an account does not have sufficient funds to start a call. Shopify uses this code when the store has not paid their fees and is temporarily disabled. Stripe uses this code for failed payments where parameters were correct, for example blocked fraudulent payments. Cloudflare Turnstile uses this code when requesting resources with cURL. x402 is an open standard that repurposes the HTTP 402 "Payment Required" status code.

403 Forbidden

The request was valid, but the server refuses action. This may be due to the user not having permission to a resource or needing an account of some sort, or attempting a prohibited action (e.g. creating a duplicate record where only one is allowed). This code is also typically used if the request provided authentication by answering the WWW-Authenticate header field challenge, but the server did not accept that authentication. The request should not be repeated.

This code differs from 401 in that while 401 is returned when the client has not authenticated, and implies that a successful response may be returned following valid authentication, 403 is returned when the client is not permitted access to the resource despite providing authentication such as insufficient permissions of the authenticated account.

The Apache web server returns 403 in response to a request for URL paths that corresponded to a file system directory when directory listing is disabled and there is no Directory Index directive to specify an existing file to be returned to the browser. Some administrators configure the Mod proxy extension to block such requests and this will also return 403. IIS responds in the same way when directory listings are denied in that server. In WebDAV, 403 is returned if the client issued a PROPFIND request but did not also issue the required Depth header or issued a Depth header of infinity.

404 Not Found

The requested resource could not be found but may be available in the future. Subsequent requests by the client are permissible.

405 Method Not Allowed

A request method is not supported for the requested resource (for example, a GET request on a form that requires data to be presented via POST, or a PUT request on a read-only resource).

406 Not Acceptable

The requested resource is capable of generating only content not acceptable according to the Accept headers sent in the request. See Content negotiation.

407 Proxy Authentication Required

The client must first authenticate itself with the proxy.

408 Request Timeout

The server timed out waiting for the request. According to HTTP specifications: "The client did not produce a request within the time that the server was prepared to wait. The client MAY repeat the request without modifications at any later time."

409 Conflict

Indicates that the request could not be processed because of conflict in the current state of the resource, such as an edit conflict between multiple simultaneous updates.

410 Gone

Indicates that the resource requested was previously in use but is no longer available and will not be available again. This should be used when a resource has been intentionally removed and the resource should be purged. Upon receiving a 410 status code, the client should not request the resource in the future. Clients such as search engines should remove the resource from their indices. Most use cases do not require clients and search engines to purge the resource, and a "404 Not Found" may be used instead.

411 Length Required

The request did not specify the length of its content, which is required by the requested resource.

412 Precondition Failed

The server does not meet one of the preconditions that the requester put on the request header fields.

413 Content Too Large

The request is larger than the server is willing or able to process. Previously called "Request Entity Too Large" and "Payload Too Large".

414 URI Too Long

The URI provided was too long for the server to process. Often the result of too much data being encoded as a query-string of a GET request, in which case it should be converted to a POST request. Called "Request-URI Too Long" previously.

415 Unsupported Media Type

The request entity has a media type which the server or resource does not support. For example, the client uploads an image as image/svg+xml, but the server requires that images use a different format.

416 Range Not Satisfiable

The client has asked for a portion of the file (byte serving), but the server cannot supply that portion. For example, if the client asked for a part of the file that lies beyond the end of the file. Called "Requested Range Not Satisfiable" previously.

417 Expectation Failed
The server cannot meet the requirements of the Expect request-header field.
418 I'm a teapot (RFC 2324, RFC 7168)
This code was defined in 1998 as one of the traditional IETF April Fools' jokes, in RFC 2324, Hyper Text Coffee Pot Control Protocol, and is not expected to be implemented by actual HTTP servers. The RFC specifies this code should be returned by teapots requested to brew coffee. This HTTP status is used as an Easter egg in some websites, such as Google.com's "I'm a teapot" easter egg. Sometimes, this status code is also used as a response to a blocked request, instead of the more appropriate 403 Forbidden.
421 Misdirected Request
The request was directed at a server that is not able to produce a response (for example because of connection reuse).
422 Unprocessable Content
The request was well-formed (i.e., syntactically correct) but could not be processed.
423 Locked (WebDAV; RFC 4918)
The resource that is being accessed is locked.
424 Failed Dependency (WebDAV; RFC 4918)
The request failed because it depended on another request and that request failed (e.g., a PROPPATCH).
425 Too Early (RFC 8470)
Indicates that the server is unwilling to risk processing a request that might be replayed.
426 Upgrade Required
The client should switch to a different protocol such as TLS/1.3, given in the Upgrade header field.
428 Precondition Required (RFC 6585)
The origin server requires the request to be conditional. Intended to prevent the 'lost update' problem, where a client GETs a resource's state, modifies it, and PUTs it back to the server, when meanwhile a third party has modified the state on the server, leading to a conflict.
429 Too Many Requests (RFC 6585)
The user has sent too many requests in a given amount of time. Intended for use with rate-limiting schemes.
431 Request Header Fields Too Large (RFC 6585)
The server is unwilling to process the request because either an individual header field, or all the header fields collectively, are too large.
451 Unavailable For Legal Reasons (RFC 7725)
A server operator has received a legal demand to deny access to a resource or to a set of resources that includes the requested resource. The code 451 was chosen as a reference to the novel Fahrenheit 451.

5xx server error

5xx status indicates that the server is aware that it has encountered an error or is otherwise incapable of performing the request. Except when responding to a HEAD request, the server should include an entity containing an explanation of the error situation, and indicate whether it is a temporary or permanent condition. Likewise, user agents should display any included entity to the user. These response codes are applicable to any request method.

500 Internal Server Error
A generic error message, given when an unexpected condition was encountered and no more specific message is suitable.
501 Not Implemented
The server either does not recognize the request method, or it lacks the ability to fulfil the request. Usually this implies future availability (e.g., a new feature of a web-service API).
502 Bad Gateway
The server was acting as a gateway or proxy and received an invalid response from the upstream server.
503 Service Unavailable
The server cannot handle the request (because it is overloaded or down for maintenance). Generally, this is a temporary state.
504 Gateway Timeout
The server was acting as a gateway or proxy and did not receive a timely response from the upstream server.
505 HTTP Version Not Supported
The server does not support the HTTP version used in the request.
506 Variant Also Negotiates (RFC 2295)
Transparent content negotiation for the request results in a circular reference.
507 Insufficient Storage (WebDAV; RFC 4918)
The server is unable to store the representation needed to complete the request.
508 Loop Detected (WebDAV; RFC 5842)
The server detected an infinite loop while processing the request (sent instead of 208 Already Reported).
510 Not Extended (RFC 2774)
Further extensions to the request are required for the server to fulfil it.
511 Network Authentication Required (RFC 6585)
The client needs to authenticate to gain network access. Intended for use by intercepting proxies used to control access to the network (e.g., "captive portals" used to require agreement to Terms of Service before granting full Internet access via a Wi-Fi hotspot).